

Sorting Techniques



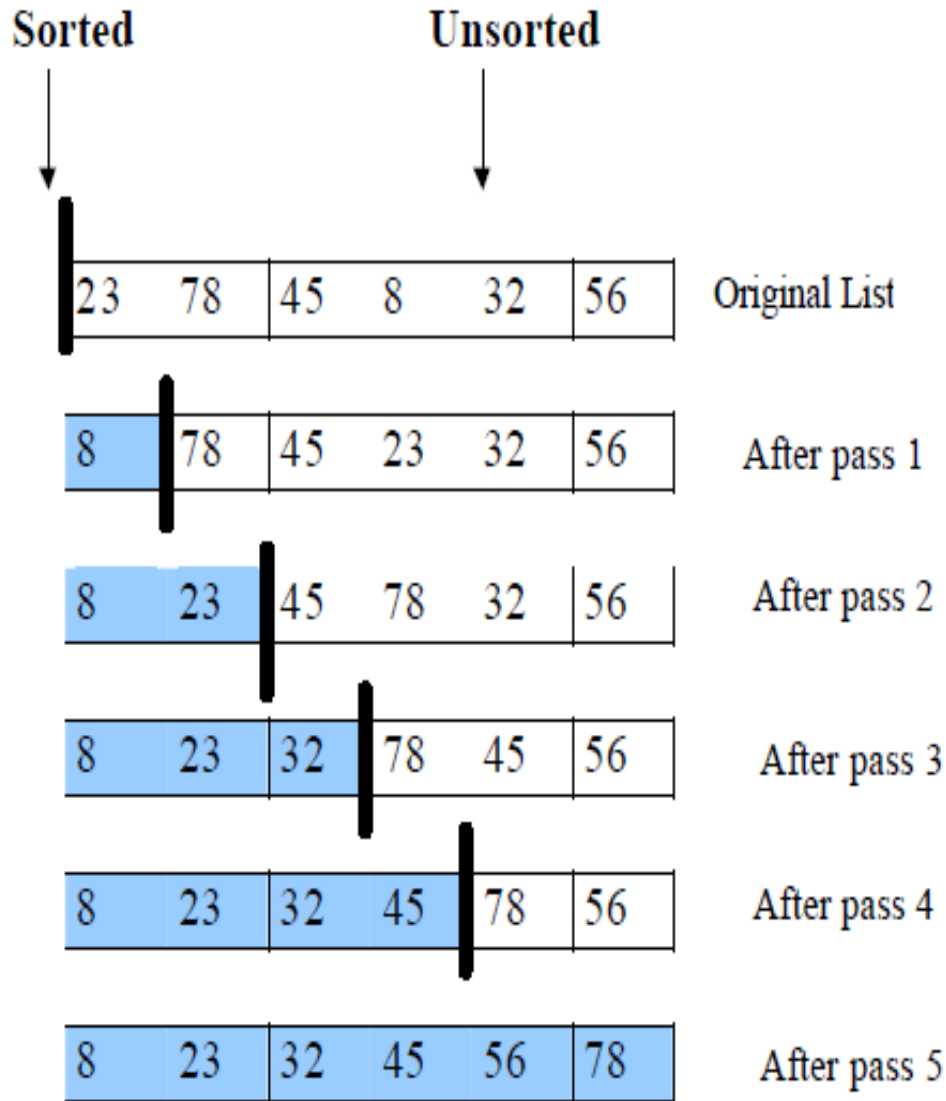
Sorting

- ▶ Sorting is the process of arranging a set of data in certain order.
- ▶ **Internal Sorting**
Deals with sorting the data held in memory of the computer
- ▶ **External Sorting**
Deals with sorting the data stored in data files, it is used when volume of data is very large and cannot be held in computers main memory

Some of the sorting algorithms are

- Selection sort
- Bubble sort
- Insertion sort
- Quick sort
- Radix sort
- Merge sort
- Bucket sort
- Tree sort

Selection Sort



Process Flow:

Read array size.
Read an array of values.
Sort the given array.
Print sorted values.

Selection Sort

- ▶ Finding smallest in each iteration

Begin

```
    set small = a[k-1]
    set loc = k-1
    for j = k to (n-1) by 1 do
        if (a[j] < small) then
            set small = a[j]
            set loc = j
        end if
    end for
```

End

```
Void SelectionSort(int a[], int n)
```

```
{
```

```
    int temp, small, loc, l, j;
```

```
    For (i=1;i<=(n-1);i++)
```

```
    {
```

```
        small = a[i-1];
```

```
        Loc=i-1;
```

```
        For (j=i; j<=(n-1);j++)
```

```
        {
```

```
            If(a[j]<small)
```

```
            {
```

```
                Small=a[j];
```

```
                Loc=j;
```

```
            }
```

```
        }
```

```
        If(loc !=(i-1) {
```

```
            temp=a[i-1];
```

```
            a[i-1]=a[loc];
```

```
            a[loc]= temp;
```

```
        }
```

```
    }
```

Selection Sort

Sorted

Unsorted



23	78	45	8	32	56
----	----	----	---	----	----

Original List

8	78	45	23	32	56
---	----	----	----	----	----

After pass 1

8	23	45	78	32	56
---	----	----	----	----	----

After pass 2

8	23	32	78	45	56
---	----	----	----	----	----

After pass 3

8	23	32	45	78	56
---	----	----	----	----	----

After pass 4

8	23	32	45	56	78
---	----	----	----	----	----

After pass 5

Analysis

$$\begin{aligned}
 F(n) &= (n-1) + (n-2) + \dots + 3 + 2 + 1 \\
 &= \frac{n(n-1)}{2} \\
 &= O(n^2)
 \end{aligned}$$

Bubble Sort

- ▶ Bubble sort is a simple and well-known sorting algorithm.
- ▶ Bubble sort belongs to $O(n^2)$ sorting algorithms, which makes it quite inefficient for sorting large data volumes.
- ▶ Bubble sort is **stable** and **adaptive**.
- ▶ **Algorithm**
 - Compare each pair of adjacent elements from the beginning of an array and, if they are in reversed order, swap them.
 - If at least one swap has been done, repeat step 1.

Bubble Sort

5	1	12	-5	16
---	---	----	----	----

unsorted

5	1	12	-5	16
---	---	----	----	----

5 > 1, swap

1	5	12	-5	16
---	---	----	----	----

5 < 12, ok

1	5	12	-5	16
---	---	----	----	----

12 > -5, swap

1	5	-5	12	16
---	---	----	----	----

12 < 16, ok

1	5	-5	12	16
---	---	----	----	----

1 < 5, ok

1	5	-5	12	16
---	---	----	----	----

5 > -5, swap

1	-5	5	12	16
---	----	---	----	----

5 < 12, ok

1	-5	5	12	16
---	----	---	----	----

1 > -5, swap

-5	1	5	12	16
----	---	---	----	----

1 < 5, ok

-5	1	5	12	16
----	---	---	----	----

-5 < 1, ok

-5	1	5	12	16
----	---	---	----	----

sorted

Bubble Sort

Begin

```

for k = 1 to (n-1) by 1 do
  for j = 0 to (n-k-1) by 1 do
    If ( a[j] > a[j+1] ) then
      temp = a [j]
      a[j]  = a [j+1]
      a[j+1]= temp
    End if
  End for
End for
End
    
```

5	1	12	-5	16
5	1	12	-5	16
1	5	12	-5	16
1	5	12	-5	16
1	5	-5	12	16
1	5	-5	12	16
1	-5	5	12	16
1	-5	5	12	16
-5	1	5	12	16
-5	1	5	12	16
-5	1	5	12	16

unsorted

5 > 1, swap

5 < 12, ok

12 > -5, swap

12 < 16, ok

1 < 5, ok

5 > -5, swap

5 < 12, ok

1 > -5, swap

1 < 5, ok

-5 < 1, ok

sorted

Analysis

$$\begin{aligned}
 F(n) &= (n-1) + (n-2) + \dots + 3 + 2 + 1 \\
 &= n(n-1) / 2 \\
 &= O(n^2)
 \end{aligned}$$

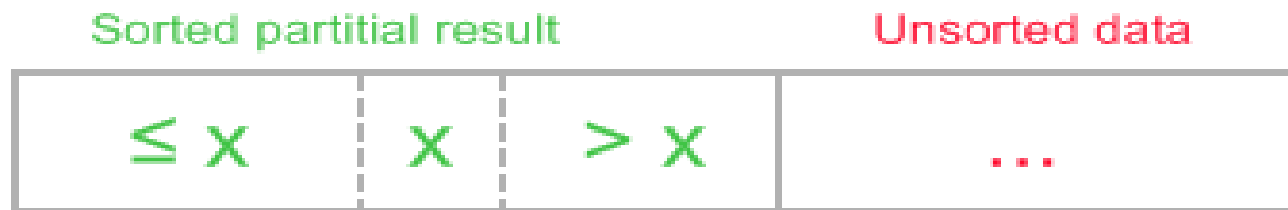
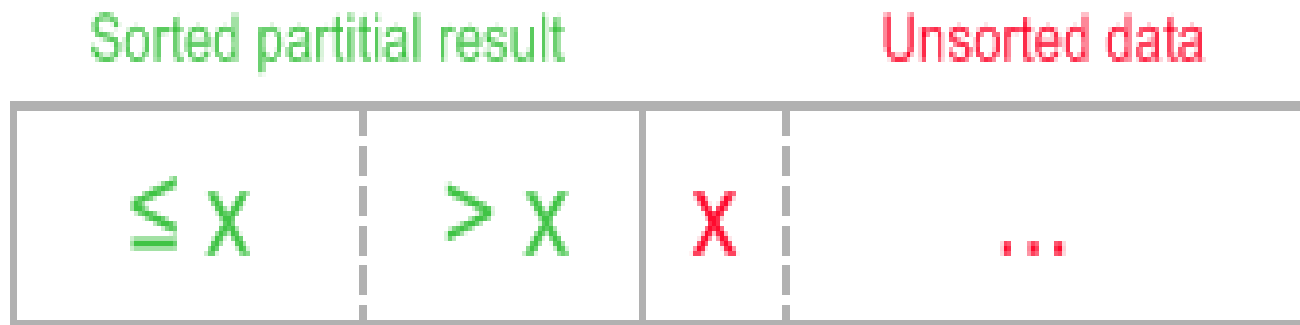
Insertion Sort

- ▶ Insertion sort belongs to the $O(n^2)$ sorting algorithms.
- ▶ Unlike many sorting algorithms with quadratic complexity, it is actually applied in practice for sorting small arrays of data.

Algorithm

- ▶ Insertion sort algorithm resembles selection sort.
- ▶ Array is imaginary divided into two parts – sorted one and unsorted one.
- ▶ At the beginning, sorted part contains **first element** of the array and unsorted one contains the rest.
- ▶ At every step, algorithm takes **first element** in the unsorted part and **inserts** it to the right place of the sorted one.
- ▶ When unsorted part becomes **empty**, algorithm *stops*.

Insertion Sort



Example: Sort {7, -5, 2, 16, 4} using insertion sort.

Insertion Sort

7	-5	2	16	4
---	----	---	----	---

unsorted

7	-5	2	16	4
---	----	---	----	---

-5 to be inserted

?	7	2	16	4
---	---	---	----	---

7 > -5, shift

-5	7	2	16	4
----	---	---	----	---

reached left boundary, insert -5

-5	7	2	16	4
----	---	---	----	---

2 to be inserted

-5	?	7	16	4
----	---	---	----	---

7 > 2, shift

-5	2	7	16	4
----	---	---	----	---

-5 < 2, insert 2

-5	2	7	16	4
----	---	---	----	---

16 to be inserted

-5	2	7	16	4
----	---	---	----	---

7 < 16, insert 16

-5	2	7	16	4
----	---	---	----	---

4 to be inserted

-5	2	7	?	16
----	---	---	---	----

16 > 4, shift

-5	2	?	7	16
----	---	---	---	----

7 > 4, shift

-5	2	4	7	16
----	---	---	---	----

2 < 4, insert 4

-5	2	4	7	16
----	---	---	---	----

sorted

Analysis

[no of comparisons]

$$F(n) = 1 + 2 + 3 + \dots + (n-3) + (n-2) + (n-1)$$

$$= n(n-1) / 2$$

$$= O(n^2)$$

Insertion Sort (A)

for j=2 to A.length

key = A[j]

//insert A[j] into sorted sequence A[1..j-1]

i=j-1

while (i>0 and A[i] > key)

A[i+1] =A [i]

i=i-1

A[i+1]= key

End for

7 -5 2 16 4

unsorted

7 -5 2 16 4

-5 to be inserted

? 7 2 16 4

7 > -5, shift

-5 7 2 16 4

reached left boundary, insert -5

-5 7 2 16 4

2 to be inserted

-5 ? 7 16 4

7 > 2, shift

-5 2 7 16 4

-5 < 2, insert 2

-5 2 7 16 4

16 to be inserted

-5 2 7 16 4

7 < 16, insert 16

-5 2 7 16 4

4 to be inserted

-5 2 7 ? 16

16 > 4, shift

-5 2 ? 7 16

7 > 4, shift

-5 2 4 7 16

2 < 4, insert 4

-5 2 4 7 16

sorted